

AI Caching Patterns: Complete Guide

Table of Contents

1. [Introduction](#)
 2. [Prompt Caching](#)
 3. [Response Caching](#)
 4. [Vector Cache](#)
 5. [Tool Cache](#)
 6. [Semantic Caching](#)
 7. [Multi-tier Architecture](#)
 8. [Implementation Examples](#)
 9. [Cost Optimization](#)
 10. [Best Practices](#)
-

Introduction

Why AI Caching Matters

Problems without caching:

- 💰 High API costs (\$0.01-\$0.10 per 1K tokens)
- ⌚ Slow response times (1-3 seconds per LLM call)
- 🔄 Redundant computations (same queries repeated)
- 📉 Poor scalability (API rate limits)

Benefits with caching:

- ✅ 80-95% cost reduction
- ✅ 10-100x faster responses
- ✅ Better user experience
- ✅ Reduced API load

Cost Comparison

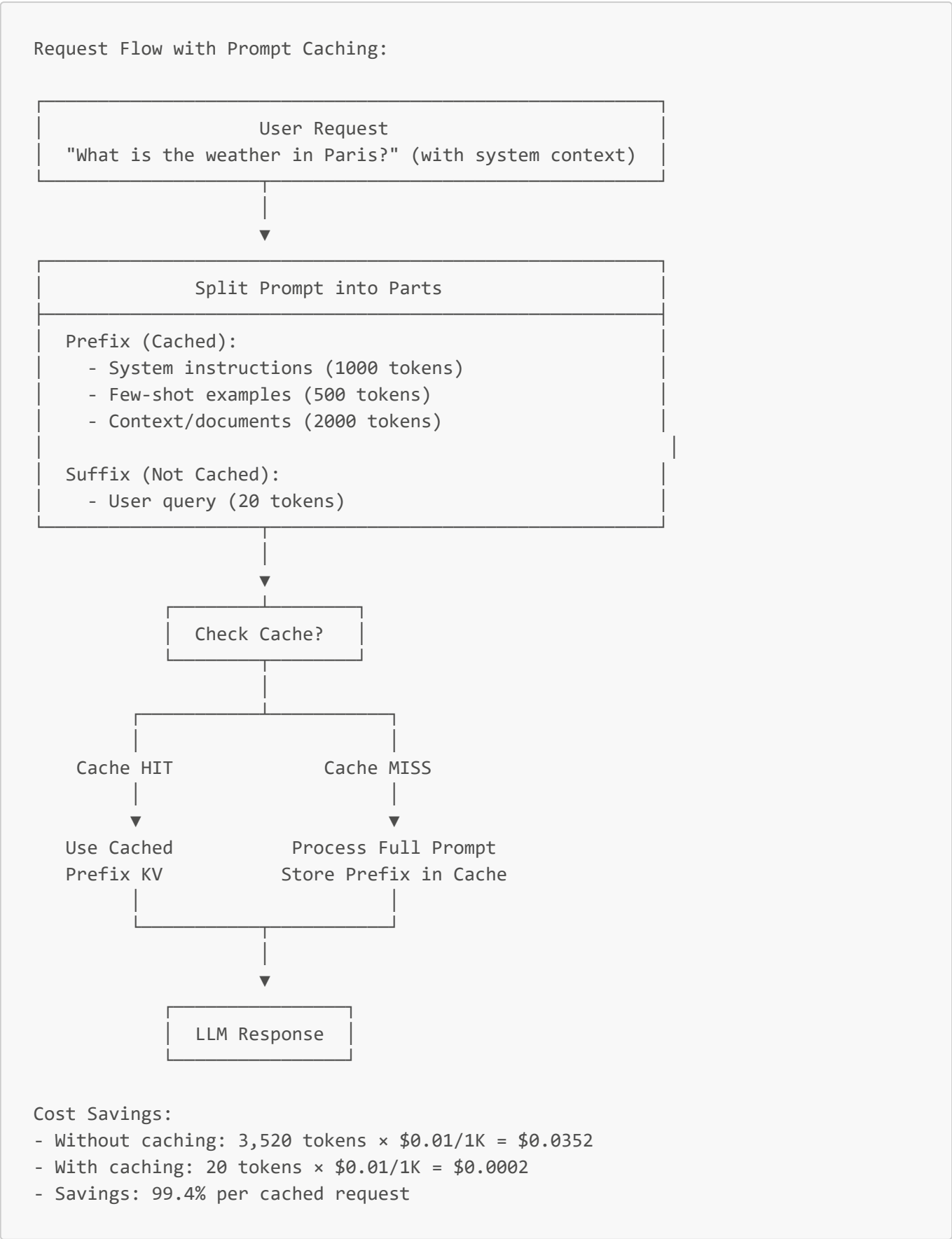
```
Without Caching (10K requests/day):  
10,000 requests × $0.002/request = $20/day = $600/month  
  
With 90% Cache Hit Rate:  
1,000 requests × $0.002/request = $2/day = $60/month  
Savings: $540/month (90% reduction)
```

1. Prompt Caching

What is Prompt Caching?

Cache the **prefix** of prompts to avoid reprocessing common context.

Architecture Diagram



Anthropic Claude Prompt Caching

How it works:

- Mark prompt sections with `cache_control`
- Prefix must be ≥ 1024 tokens
- Cache TTL: 5 minutes
- 90% cost reduction on cached tokens

Example:

```
import anthropic

client = anthropic.Anthropic(api_key="key")

# Large system context (will be cached)
system_context = """
You are a helpful customer service agent with access to:
- Product catalog (3000 products)
- Company policies (50 pages)
- FAQ database (500 questions)
[... 1500 tokens of context ...]
"""

message = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=1024,
    system=[
        {
            "type": "text",
            "text": system_context,
            "cache_control": {"type": "ephemeral"} # Mark for caching
        }
    ],
    messages=[
        {"role": "user", "content": "What is your return policy?"}
    ]
)

# Next request reuses cached system context
message2 = client.messages.create(
    model="claude-3-5-sonnet-20241022",
    max_tokens=1024,
    system=[
        {
            "type": "text",
            "text": system_context,
            "cache_control": {"type": "ephemeral"} # Cache hit!
        }
    ],
    messages=[
        {"role": "user", "content": "Do you ship internationally?"}
    ]
)
```

```
print(f"Cache hit! Saved {message2.usage.cache_read_input_tokens} tokens")
```

OpenAI Prompt Caching (GPT-4 Turbo+)

```
from openai import OpenAI

client = OpenAI()

# System message cached automatically if repeated
system_message = {
    "role": "system",
    "content": "You are an expert Python developer..." # Long context
}

# First call - cache miss
response1 = client.chat.completions.create(
    model="gpt-4-turbo",
    messages=[
        system_message,
        {"role": "user", "content": "Write a function to sort a list"}
    ]
)

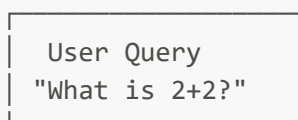
# Second call - cache hit (same system message)
response2 = client.chat.completions.create(
    model="gpt-4-turbo",
    messages=[
        system_message, # Cached!
        {"role": "user", "content": "Write a function to reverse a string"}
    ]
)
```

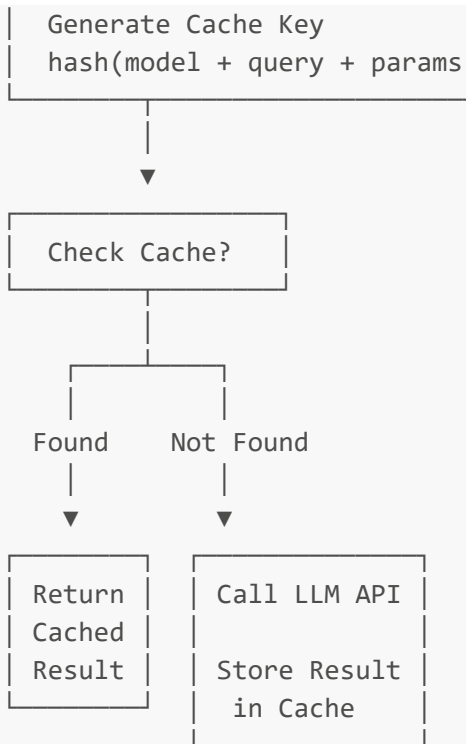
2. Response Caching

What is Response Caching?

Cache complete LLM responses for **identical** queries.

Decision Flowchart





Cache Key Structure:

```

{
  "model": "gpt-4",
  "messages": [...],
  "temperature": 0.0, # Important! Only cache temp=0
  "max_tokens": 1000
}
  
```

Implementation with Redis

```

import hashlib
import json
import redis
from openai import OpenAI

redis_client = redis.Redis(host='localhost', port=6379, decode_responses=True)
openai_client = OpenAI()

class LLMCache:
    def __init__(self, ttl=3600):
        self.ttl = ttl

    def _generate_cache_key(self, model, messages, **kwargs):
        """Generate deterministic cache key"""
        cache_dict = {
            "model": model,
            "messages": messages,
            **kwargs
        }
        cache_str = json.dumps(cache_dict, sort_keys=True)
  
```

```

        return f"llm:cache:{hashlib.sha256(cache_str.encode()).hexdigest()}"

def get_completion(self, model, messages, temperature=0.0, **kwargs):
    """Get completion with caching"""

    # Only cache deterministic responses (temperature=0)
    if temperature != 0.0:
        return self._call_llm(model, messages, temperature, **kwargs)

    # Generate cache key
    cache_key = self._generate_cache_key(model, messages,
    temperature=temperature, **kwargs)

    # Check cache
    cached = redis_client.get(cache_key)
    if cached:
        print("✓ Cache HIT")
        return json.loads(cached)

    # Cache miss - call LLM
    print("X Cache MISS - Calling LLM")
    response = self._call_llm(model, messages, temperature, **kwargs)

    # Store in cache
    redis_client.setex(
        cache_key,
        self.ttl,
        json.dumps(response)
    )

    return response

def _call_llm(self, model, messages, temperature, **kwargs):
    """Actual LLM API call"""
    completion = openai_client.chat.completions.create(
        model=model,
        messages=messages,
        temperature=temperature,
        **kwargs
    )
    return completion.choices[0].message.content

# Usage
cache = LLMCache(ttl=3600)

# First call - cache miss
response1 = cache.get_completion(
    model="gpt-4",
    messages=[{"role": "user", "content": "What is 2+2?"}],
    temperature=0.0
)

# Second call - cache hit!
response2 = cache.get_completion(

```

```

    model="gpt-4",
    messages=[{"role": "user", "content": "What is 2+2?"}],
    temperature=0.0
)

```

Cache Decorator Pattern

```

from functools import wraps
import hashlib
import json

def llm_cache(ttl=3600):
    """Decorator for caching LLM function calls"""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            # Generate cache key from function args
            cache_data = {
                'func': func.__name__,
                'args': args,
                'kwargs': kwargs
            }
            cache_key = f"llm:{hashlib.sha256(json.dumps(cache_data,
sort_keys=True).encode()).hexdigest()}"

            # Check cache
            cached = redis_client.get(cache_key)
            if cached:
                return json.loads(cached)

            # Call function
            result = func(*args, **kwargs)

            # Store result
            redis_client.setex(cache_key, ttl, json.dumps(result))

            return result
        return wrapper
    return decorator

# Usage
@llm_cache(ttl=3600)
def ask_question(question: str) -> str:
    response = openai_client.chat.completions.create(
        model="gpt-4",
        messages=[{"role": "user", "content": question}],
        temperature=0.0
    )
    return response.choices[0].message.content

```

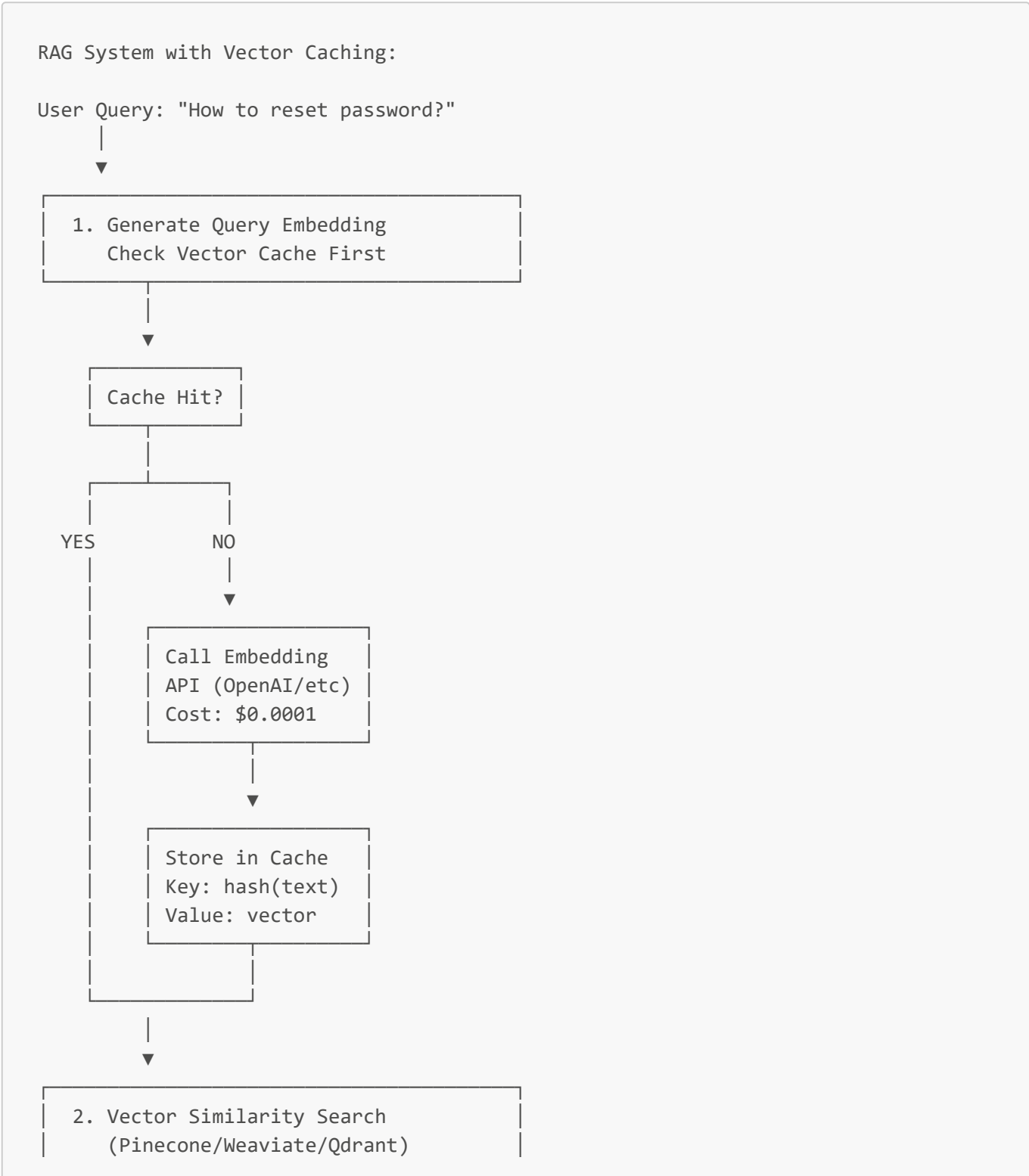
```
# Automatically cached
answer = ask_question("What is the capital of France?")
```

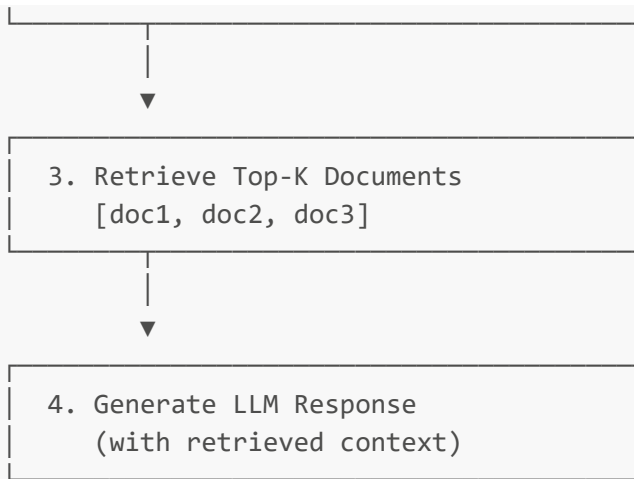
3. Vector Cache (Embedding Cache)

What is Vector Caching?

Cache vector embeddings to avoid recomputing expensive embedding operations.

Architecture Diagram





Savings per Query:

- Embedding API call: \$0.0001
- With 80% cache hit: \$0.00002
- At 1M queries/month: Save \$80

Implementation with Redis

```

import hashlib
import numpy as np
import redis
from openai import OpenAI

redis_client = redis.Redis(host='localhost', port=6379)
openai_client = OpenAI()

class VectorCache:
    def __init__(self, ttl=86400): # 24 hours
        self.ttl = ttl
        self.embedding_model = "text-embedding-3-small"

    def _hash_text(self, text: str) -> str:
        """Generate cache key from text"""
        return f"emb:{hashlib.sha256(text.encode()).hexdigest()}"

    def get_embedding(self, text: str) -> list[float]:
        """Get embedding with caching"""
        cache_key = self._hash_text(text)

        # Check cache
        cached = redis_client.get(cache_key)
        if cached:
            print("✓ Vector Cache HIT")
            return np.frombuffer(cached, dtype=np.float32).tolist()

        # Cache miss - generate embedding
        print("X Vector Cache MISS - Calling Embedding API")
        response = openai_client.embeddings.create(

```

```

        model=self.embedding_model,
        input=text
    )
    embedding = response.data[0].embedding

    # Store in cache (as bytes for efficiency)
    redis_client.setex(
        cache_key,
        self.ttl,
        np.array(embedding, dtype=np.float32).tobytes()
    )

    return embedding

# Usage in RAG
vector_cache = VectorCache()

# First query - cache miss
query1 = "How to reset my password?"
embedding1 = vector_cache.get_embedding(query1) # API call

# Same query later - cache hit!
embedding2 = vector_cache.get_embedding(query1) # From cache

# Similar query - cache miss (different text)
query2 = "How do I change my password?"
embedding3 = vector_cache.get_embedding(query2) # API call

```

Document Embedding Cache

```

class DocumentVectorCache:
    """Cache embeddings for document chunks"""

    def __init__(self):
        self.cache = {}

    def embed_documents(self, documents: list[str]) -> list[list[float]]:
        """Batch embed with caching"""
        embeddings = []
        to_embed = []
        to_embed_indices = []

        # Check cache for each document
        for i, doc in enumerate(documents):
            cache_key = self._hash_text(doc)
            cached = redis_client.get(cache_key)

            if cached:
                # Cache hit
                embeddings.append(np.frombuffer(cached,
                    dtype=np.float32).tolist())

```

```

        else:
            # Cache miss - need to embed
            embeddings.append(None)
            to_embed.append(doc)
            to_embed_indices.append(i)

# Batch embed uncached documents
if to_embed:
    response = openai_client.embeddings.create(
        model="text-embedding-3-small",
        input=to_embed
    )

    # Store and update
    for idx, embedding_obj in zip(to_embed_indices, response.data):
        embedding = embedding_obj.embedding
        embeddings[idx] = embedding

        # Cache it
        cache_key = self._hash_text(documents[idx])
        redis_client.setex(
            cache_key,
            86400,
            np.array(embedding, dtype=np.float32).tobytes()
        )

    print(f"Cached: {len(documents) - len(to_embed)}, Computed: {len(to_embed)}")
    return embeddings

```

Semantic Deduplication

```

from sklearn.metrics.pairwise import cosine_similarity

class SemanticDeduplicator:
    """Remove semantically similar documents using vector cache"""

    def __init__(self, similarity_threshold=0.95):
        self.threshold = similarity_threshold
        self.vector_cache = VectorCache()

    def deduplicate(self, documents: list[str]) -> list[str]:
        """Remove near-duplicate documents"""
        if not documents:
            return []

        # Get embeddings (with caching)
        embeddings = [self.vector_cache.get_embedding(doc) for doc in documents]
        embeddings_array = np.array(embeddings)

        # Compute similarity matrix

```

```

        similarities = cosine_similarity(embeddings_array)

        # Keep only unique documents
        kept_indices = []
        for i in range(len(documents)):
            is_duplicate = False
            for j in kept_indices:
                if similarities[i][j] > self.threshold:
                    is_duplicate = True
                    break

            if not is_duplicate:
                kept_indices.append(i)

        return [documents[i] for i in kept_indices]

# Usage
deduplicator = SemanticDeduplicator()

docs = [
    "How to reset password",
    "Password reset instructions", # Similar - will be removed
    "Account billing questions"
]

unique_docs = deduplicator.deduplicate(docs)
# Result: ["How to reset password", "Account billing questions"]

```

4. Tool Cache (Function Call Result Caching)

What is Tool Caching?

Cache results of **deterministic** tool/function calls to avoid redundant executions.

Decision Flowchart

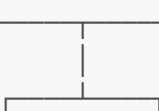
Agent receives tool call request

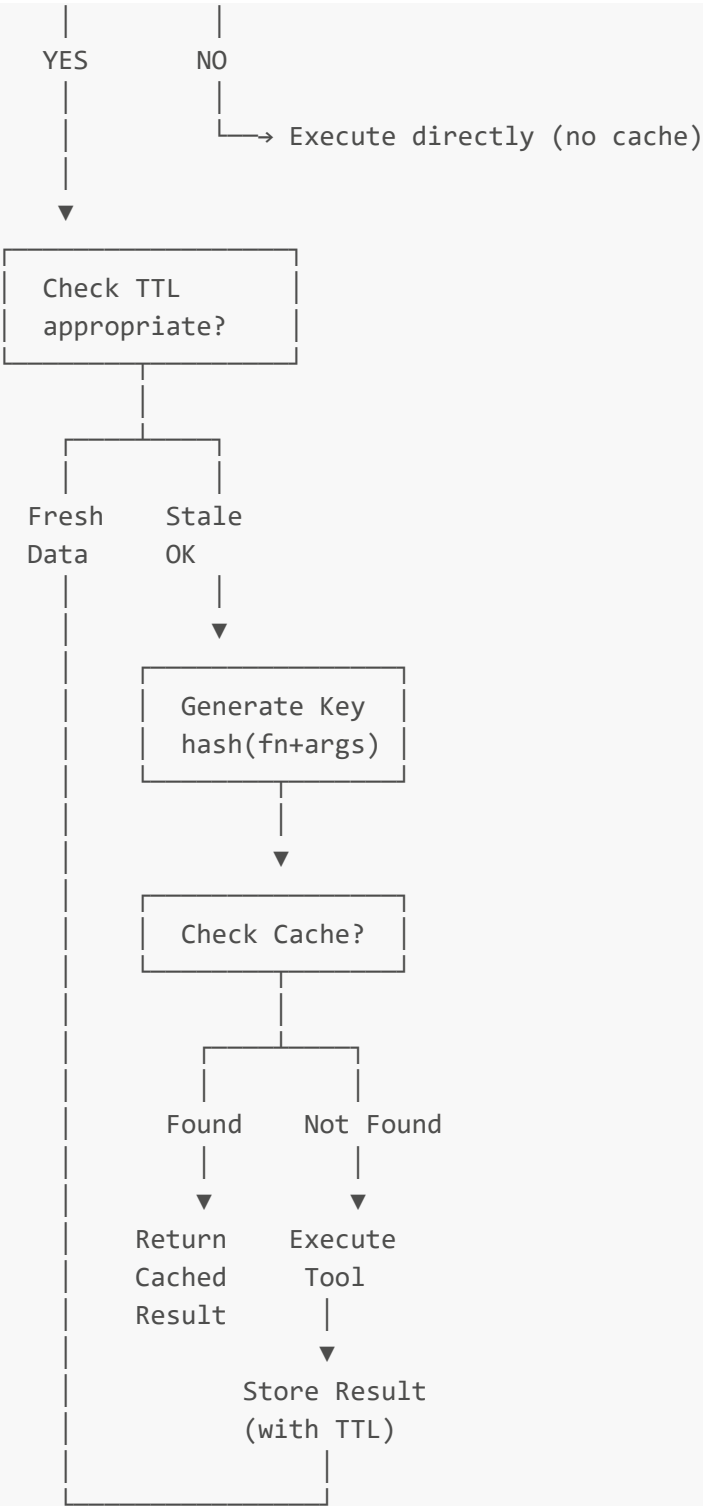


Tool: get_weather(city="Paris")



Is tool deterministic?
(same input → same output)





Tool Caching Decision Table:

Tool	Deterministic	TTL	Cache?
get_weather()	No (changes)	10 min	YES (short)
calculate_sum()	Yes	Forever	YES
get_stock_price()	No (changes)	1 min	YES (short)
send_email()	Side-effect	N/A	NO
get_user_profile()	Yes (mostly)	1 hour	YES
random_number()	No (random)	N/A	NO

Implementation with LangChain

```

import hashlib
import json
import redis
from typing import Any, Callable
from functools import wraps

redis_client = redis.Redis(host='localhost', port=6379, decode_responses=True)

class ToolCache:
    """Cache tool execution results"""

    def __init__(self, default_ttl=3600):
        self.default_ttl = default_ttl

    def cached_tool(self, ttl=None):
        """Decorator for caching tool results"""
        ttl = ttl or self.default_ttl

        def decorator(func: Callable) -> Callable:
            @wraps(func)
            def wrapper(*args, **kwargs) -> Any:
                # Generate cache key
                cache_data = {
                    'tool': func.__name__,
                    'args': args,
                    'kwargs': kwargs
                }
                cache_key = f"tool:{hashlib.sha256(json.dumps(cache_data,
sort_keys=True).encode()).hexdigest()}"

                # Check cache
                cached = redis_client.get(cache_key)
                if cached:
                    print(f"✓ Tool Cache HIT: {func.__name__}")
                    return json.loads(cached)

                # Cache miss - execute tool
                print(f"X Tool Cache MISS: {func.__name__} - Executing...")
                result = func(*args, **kwargs)

                # Store result
                redis_client.setex(cache_key, ttl, json.dumps(result))

                return result
            return wrapper
        return decorator

tool_cache = ToolCache()

# Example tools

```

```

@tool_cache.cached_tool(ttl=600) # 10 minutes
def get_weather(city: str) -> dict:
    """Get weather for a city (expensive API call)"""
    import requests
    # Simulated API call
    response = requests.get(f"https://api.weather.com/v1/current?city={city}")
    return response.json()

@tool_cache.cached_tool(ttl=3600) # 1 hour
def get_stock_price(symbol: str) -> float:
    """Get current stock price"""
    import yfinance as yf
    stock = yf.Ticker(symbol)
    return stock.info['currentPrice']

@tool_cache.cached_tool(ttl=None) # Cache forever
def calculate_compound_interest(principal: float, rate: float, years: int) -> float:
    """Calculate compound interest (pure function)"""
    return principal * (1 + rate) ** years

# Non-cacheable (side effects)
def send_email(to: str, subject: str, body: str) -> bool:
    """Send email - DO NOT CACHE"""
    # Email sending logic
    return True

# Usage
weather1 = get_weather("Paris") # Cache miss - API call
weather2 = get_weather("Paris") # Cache hit - instant
weather3 = get_weather("London") # Cache miss - different city

stock1 = get_stock_price("AAPL") # Cache miss - API call
stock2 = get_stock_price("AAPL") # Cache hit - instant

interest = calculate_compound_interest(1000, 0.05, 10) # Cache hit after first call

```

LangChain Agent with Tool Caching

```

from langchain.agents import Tool, AgentExecutor, create_openai_functions_agent
from langchain.chat_models import ChatOpenAI
from langchain.prompts import ChatPromptTemplate, MessagesPlaceholder

# Define tools with caching
@tool_cache.cached_tool(ttl=600)
def search_database(query: str) -> str:
    """Search internal database"""
    # Expensive database query
    results = database.search(query)
    return json.dumps(results)

```

```

@tool_cache.cached_tool(ttl=300)
def get_user_info(user_id: str) -> dict:
    """Get user information"""
    # API call to user service
    user = user_service.get_user(user_id)
    return user

# Create tools
tools = [
    Tool(
        name="SearchDatabase",
        func=search_database,
        description="Search the database for information"
    ),
    Tool(
        name="GetUserInfo",
        func=get_user_info,
        description="Get user information by ID"
    )
]

# Create agent
llm = ChatOpenAI(model="gpt-4", temperature=0)
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant with access to tools."),
    MessagesPlaceholder(variable_name="chat_history", optional=True),
    ("human", "{input}"),
    MessagesPlaceholder(variable_name="agent_scratchpad"),
])

agent = create_openai_functions_agent(llm, tools, prompt)
agent_executor = AgentExecutor(agent=agent, tools=tools, verbose=True)

# Multiple queries about same user - tool cache helps
response1 = agent_executor.invoke({"input": "What is user 123's email?"})
# Tool call: get_user_info("123") - Cache MISS

response2 = agent_executor.invoke({"input": "What is user 123's phone number?"})
# Tool call: get_user_info("123") - Cache HIT!

```

Smart Tool Cache with Invalidation

```

class SmartToolCache:
    """Tool cache with smart invalidation"""

    def __init__(self):
        self.cache_dependencies = {} # Track what invalidates what

    def register_dependency(self, tool_name: str, invalidates: list[str]):
        """Register that calling tool_name invalidates other tools"""

```

```

        self.cache_dependencies[tool_name] = invalidates

    def invalidate_related(self, tool_name: str):
        """Invalidate caches related to this tool"""
        if tool_name in self.cache_dependencies:
            for related_tool in self.cache_dependencies[tool_name]:
                pattern = f"tool:*{related_tool}*"
                for key in redis_client.scan_iter(match=pattern):
                    redis_client.delete(key)
                print(f"Invalidated cache for: {related_tool}")

    def cached_tool_with_invalidation(self, ttl=3600):
        """Decorator with automatic invalidation"""
        def decorator(func: Callable) -> Callable:
            @wraps(func)
            def wrapper(*args, **kwargs) -> Any:
                # Check if this is a write operation
                is_write_op = func.__name__.startswith(('update_', 'delete_',
'create_'))

                if is_write_op:
                    # Execute without caching
                    result = func(*args, **kwargs)
                    # Invalidate related caches
                    self.invalidate_related(func.__name__)
                    return result

                # Read operation - use cache
                cache_key = self._generate_key(func, args, kwargs)
                cached = redis_client.get(cache_key)

                if cached:
                    return json.loads(cached)

                result = func(*args, **kwargs)
                redis_client.setex(cache_key, ttl, json.dumps(result))
                return result
            return wrapper
        return decorator

smart_cache = SmartToolCache()

# Register dependencies
smart_cache.register_dependency('update_user_email', ['get_user_info',
'search_users'])

@smart_cache.cached_tool_with_invalidation(ttl=3600)
def get_user_info(user_id: str) -> dict:
    """Get user info - cacheable"""
    return database.get_user(user_id)

@smart_cache.cached_tool_with_invalidation()
def update_user_email(user_id: str, new_email: str) -> bool:
    """Update user email - invalidates get_user_info cache"""

```

```

    result = database.update_user(user_id, {'email': new_email})
    return result

# Usage
user1 = get_user_info("123")          # Cache miss
user2 = get_user_info("123")          # Cache hit

update_user_email("123", "new@example.com") # Invalidates cache
user3 = get_user_info("123")          # Cache miss (invalidated)

```

API Rate Limit Protection with Cache

```

class RateLimitedToolCache:
    """Cache with rate limit protection"""

    def __init__(self, rate_limit: int, time_window: int):
        """
        rate_limit: max calls per time_window
        time_window: in seconds
        """
        self.rate_limit = rate_limit
        self.time_window = time_window

    def rate_limited_tool(self, ttl=3600):
        def decorator(func: Callable) -> Callable:
            @wraps(func)
            def wrapper(*args, **kwargs) -> Any:
                # Check cache first
                cache_key = self._generate_key(func, args, kwargs)
                cached = redis_client.get(cache_key)
                if cached:
                    return json.loads(cached)

                # Check rate limit
                rate_key = f"rate:{func.__name__}"
                current_count = redis_client.get(rate_key)

                if current_count and int(current_count) >= self.rate_limit:
                    raise Exception(f"Rate limit exceeded for {func.__name__}")

                # Execute function
                result = func(*args, **kwargs)

                # Update rate limit counter
                if not current_count:
                    redis_client.setex(rate_key, self.time_window, 1)
                else:
                    redis_client.incr(rate_key)

                # Cache result
                redis_client.setex(cache_key, ttl, json.dumps(result))

```

```

        return result
    return wrapper
return decorator

# Example: API with 100 calls/hour limit
rate_limited_cache = RateLimitedToolCache(rate_limit=100, time_window=3600)

@rate_limited_cache.rate_limited_tool(ttl=600)
def call_expensive_api(param: str) -> dict:
    """API with rate limits"""
    response = requests.get(f"https://api.example.com/data?param={param}")
    return response.json()

# Make 150 calls - first 100 execute, rest use cache or fail gracefully
for i in range(150):
    try:
        result = call_expensive_api(f"query_{i % 50}") # Repeated queries
    except Exception as e:
        print(f"Rate limited: {e}")

```

5. Semantic Caching

What is Semantic Caching?

Cache responses for **semantically similar** queries (not just exact matches).

Architecture

Traditional Cache (Exact Match):

Query: "What is machine learning?" → Cache Key: hash("What is machine learning?")

Query: "Explain machine learning" → Different hash → Cache MISS X

Semantic Cache (Similarity Match):

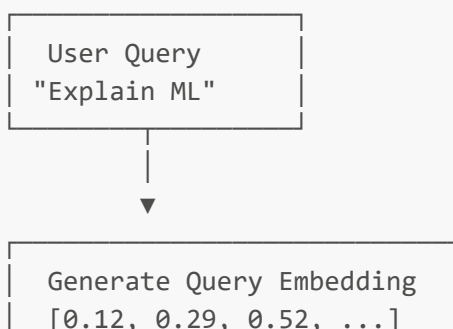
Query: "What is machine learning?"

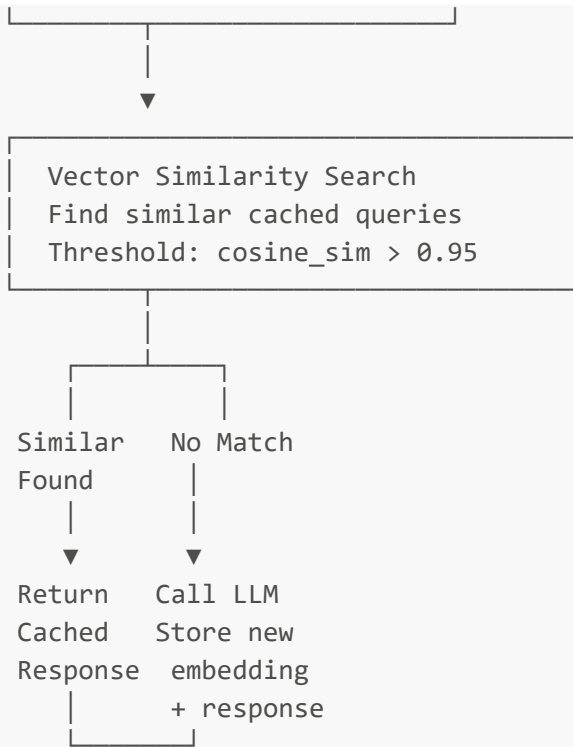
→ Embed → [0.1, 0.3, 0.5, ...] → Store with response

Query: "Explain machine learning"

→ Embed → [0.12, 0.29, 0.52, ...] → Similarity: 0.97 → Cache HIT ✓

Flow Diagram:





Implementation with Pinecone

```

import pinecone
from openai import OpenAI
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

openai_client = OpenAI()
pinecone.init(api_key="your-key", environment="us-west1-gcp")

class SemanticCache:
    def __init__(self, index_name="semantic-cache", similarity_threshold=0.95):
        self.index = pinecone.Index(index_name)
        self.threshold = similarity_threshold
        self.embedding_model = "text-embedding-3-small"

    def _get_embedding(self, text: str) -> list[float]:
        """Generate embedding for text"""
        response = openai_client.embeddings.create(
            model=self.embedding_model,
            input=text
        )
        return response.data[0].embedding

    def get_response(self, query: str, llm_func: callable) -> str:
        """Get LLM response with semantic caching"""

        # Generate query embedding
        query_embedding = self._get_embedding(query)

```

```

# Search for similar queries in cache
results = self.index.query(
    vector=query_embedding,
    top_k=1,
    include_metadata=True
)

# Check if similar query exists
if results['matches'] and results['matches'][0]['score'] > self.threshold:
    # Cache hit!
    cached_response = results['matches'][0]['metadata']['response']
    cached_query = results['matches'][0]['metadata']['query']
    similarity = results['matches'][0]['score']

    print(f"✓ Semantic Cache HIT!")
    print(f"  Original: {cached_query}")
    print(f"  Current:  {query}")
    print(f"  Similarity: {similarity:.3f}")

    return cached_response

# Cache miss - call LLM
print("X Semantic Cache MISS - Calling LLM")
response = llm_func(query)

# Store in cache
import uuid
self.index.upsert(vectors=[{
    'id': str(uuid.uuid4()),
    'values': query_embedding,
    'metadata': {
        'query': query,
        'response': response
    }
}])

return response

# Usage
cache = SemanticCache(similarity_threshold=0.95)

def ask_gpt(query: str) -> str:
    """Call GPT-4"""
    response = openai_client.chat.completions.create(
        model="gpt-4",
        messages=[{"role": "user", "content": query}],
        temperature=0.0
    )
    return response.choices[0].message.content

# First query
response1 = cache.get_response(
    "What is machine learning?",
    ask_gpt

```

```

)
# Cache MISS - calls LLM

# Similar query - cache hit!
response2 = cache.get_response(
    "Explain machine learning to me",
    ask_gpt
)
# Cache HIT - returns cached response

# Very similar query
response3 = cache.get_response(
    "Can you explain what ML is?",
    ask_gpt
)
# Cache HIT - same cached response

```

In-Memory Semantic Cache (Lightweight)

```

class InMemorySemanticCache:
    """Lightweight semantic cache without external vector DB"""

    def __init__(self, similarity_threshold=0.95, max_size=1000):
        self.cache = [] # List of (embedding, query, response)
        self.threshold = similarity_threshold
        self.max_size = max_size

    def _cosine_similarity(self, vec1: list, vec2: list) -> float:
        """Calculate cosine similarity"""
        return np.dot(vec1, vec2) / (np.linalg.norm(vec1) * np.linalg.norm(vec2))

    def get(self, query: str, llm_func: callable) -> str:
        """Get response with semantic caching"""

        # Generate embedding
        query_emb = self._get_embedding(query)

        # Search cache
        best_match = None
        best_similarity = 0

        for cached_emb, cached_query, cached_response in self.cache:
            similarity = self._cosine_similarity(query_emb, cached_emb)

            if similarity > best_similarity:
                best_similarity = similarity
                best_match = (cached_query, cached_response)

        # Check threshold
        if best_match and best_similarity > self.threshold:
            print(f"✓ Semantic hit (similarity: {best_similarity:.3f})")

```

```
        return best_match[1]

    # Cache miss
    print("X Semantic miss - calling LLM")
    response = llm_func(query)

    # Add to cache
    self.cache.append((query_emb, query, response))

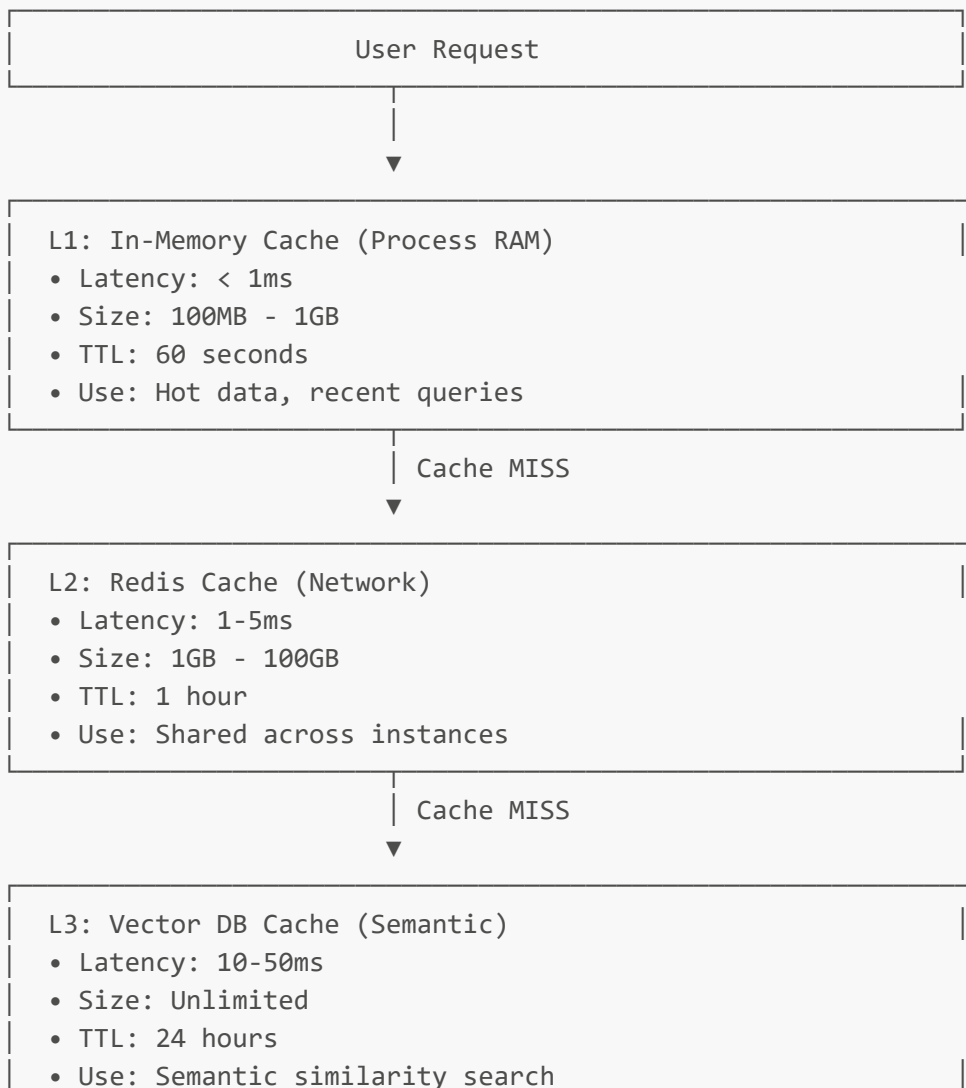
    # Enforce max size (LRU)
    if len(self.cache) > self.max_size:
        self.cache.pop(0)

    return response
```

6. Multi-tier Caching Architecture

Architecture Diagram

Multi-tier Cache Hierarchy (Fastest → Slowest):



Cache MISS

- Source: LLM API Call
- Latency: 1-3 seconds
 - Cost: \$\$\$
 - Store result in L1, L2, L3

Performance Comparison:

Tier	Latency	Hit Rate	Cost
L1	0.5ms	20%	Free
L2	3ms	50%	\$
L3	30ms	20%	\$\$
LLM	2000ms	10%	\$\$\$\$

Overall cache hit: 90%

Average latency: $0.2 \times 0.5 + 0.5 \times 3 + 0.2 \times 30 + 0.1 \times 2000 = 208\text{ms}$

vs No cache: 2000ms (10x improvement)

Implementation

```
from cachetools import LRUCache
import redis
import pinecone
from openai import OpenAI
import time

class MultiTierCache:
    """Three-tier caching system"""

    def __init__(self):
        # L1: In-memory LRU cache
        self.l1_cache = LRUCache(maxsize=1000) # 1000 entries

        # L2: Redis
        self.l2_cache = redis.Redis(host='localhost', port=6379,
decode_responses=True)

        # L3: Vector DB (Pinecone)
        self.l3_cache = pinecone.Index("semantic-cache")

        self.openai = OpenAI()
        self.stats = {
            'l1_hits': 0,
            'l2_hits': 0,
            'l3_hits': 0,
```

```

        'llm_calls': 0
    }

def get(self, query: str) -> tuple[str, str]:
    """Get response with multi-tier caching"""
    import hashlib
    cache_key = hashlib.sha256(query.encode()).hexdigest()

    # L1: Check in-memory cache
    if cache_key in self.l1_cache:
        self.stats['l1_hits'] += 1
        print("✓ L1 Cache HIT (in-memory)")
        return self.l1_cache[cache_key], 'L1'

    # L2: Check Redis
    l2_result = self.l2_cache.get(f"cache:{cache_key}")
    if l2_result:
        self.stats['l2_hits'] += 1
        print("✓ L2 Cache HIT (Redis)")
        # Promote to L1
        self.l1_cache[cache_key] = l2_result
        return l2_result, 'L2'

    # L3: Check semantic cache (Pinecone)
    query_embedding = self._get_embedding(query)
    l3_results = self.l3_cache.query(
        vector=query_embedding,
        top_k=1,
        include_metadata=True
    )

    if l3_results['matches'] and l3_results['matches'][0]['score'] > 0.95:
        self.stats['l3_hits'] += 1
        response = l3_results['matches'][0]['metadata']['response']
        print(f"✓ L3 Cache HIT (Semantic, similarity: {l3_results['matches']
[0]['score']:.3f})")

        # Promote to L2 and L1
        self.l2_cache.setex(f"cache:{cache_key}", 3600, response)
        self.l1_cache[cache_key] = response
        return response, 'L3'

    # Cache MISS - call LLM
    self.stats['llm_calls'] += 1
    print("X ALL Cache MISS - Calling LLM")
    response = self._call_llm(query)

    # Store in all tiers
    self.l1_cache[cache_key] = response
    self.l2_cache.setex(f"cache:{cache_key}", 3600, response)
    self._store_semantic(query, query_embedding, response)

    return response, 'LLM'

```

```

def _call_llm(self, query: str) -> str:
    """Call LLM API"""
    response = self.openai.chat.completions.create(
        model="gpt-4",
        messages=[{"role": "user", "content": query}],
        temperature=0.0
    )
    return response.choices[0].message.content

def _get_embedding(self, text: str) -> list[float]:
    """Generate embedding"""
    response = self.openai.embeddings.create(
        model="text-embedding-3-small",
        input=text
    )
    return response.data[0].embedding

def _store_semantic(self, query: str, embedding: list, response: str):
    """Store in semantic cache"""
    import uuid
    self.l3_cache.upsert(vectors=[{
        'id': str(uuid.uuid4()),
        'values': embedding,
        'metadata': {'query': query, 'response': response}
    }])

def get_stats(self) -> dict:
    """Get cache statistics"""
    total = sum(self.stats.values())
    return {
        **self.stats,
        'total_requests': total,
        'overall_hit_rate': (total - self.stats['llm_calls']) / total if total
> 0 else 0
    }

# Usage
cache = MultiTierCache()

# Benchmark
queries = [
    "What is Python?",
    "What is Python?", # Exact match - L1 hit
    "Explain Python programming", # Similar - L3 hit
    "What is JavaScript?", # New query - LLM call
]

for i, query in enumerate(queries):
    print(f"\n--- Query {i+1}: {query} ---")
    start = time.time()
    response, tier = cache.get(query)
    latency = (time.time() - start) * 1000
    print(f"Latency: {latency:.2f}ms | Tier: {tier}")

```

```
print("\n--- Cache Statistics ---")
print(cache.get_stats())
```

7. Cost Optimization Strategies

ROI Calculator

```
class CacheROI:
    """Calculate ROI of caching strategy"""

    def __init__(self,
                  requests_per_day: int,
                  avg_prompt_tokens: int = 500,
                  avg_completion_tokens: int = 200,
                  model_cost_per_1k_input: float = 0.01,
                  model_cost_per_1k_output: float = 0.03,
                  cache_cost_per_month: float = 50):

        self.requests_per_day = requests_per_day
        self.avg_prompt_tokens = avg_prompt_tokens
        self.avg_completion_tokens = avg_completion_tokens
        self.input_cost = model_cost_per_1k_input
        self.output_cost = model_cost_per_1k_output
        self.cache_cost = cache_cost_per_month

    def calculate(self, cache_hit_rate: float) -> dict:
        """Calculate costs and savings"""

        # Monthly requests
        monthly_requests = self.requests_per_day * 30

        # Cost without cache
        cost_per_request = (
            (self.avg_prompt_tokens / 1000) * self.input_cost +
            (self.avg_completion_tokens / 1000) * self.output_cost
        )
        cost_without_cache = monthly_requests * cost_per_request

        # Cost with cache
        cache_misses = monthly_requests * (1 - cache_hit_rate)
        llm_cost_with_cache = cache_misses * cost_per_request
        total_cost_with_cache = llm_cost_with_cache + self.cache_cost

        # Savings
        savings = cost_without_cache - total_cost_with_cache
        savings_pct = (savings / cost_without_cache) * 100

        # ROI
        roi = ((savings - self.cache_cost) / self.cache_cost) * 100
```

```

        return {
            'monthly_requests': monthly_requests,
            'cost_without_cache': f"${cost_without_cache:.2f}",
            'cost_with_cache': f"${total_cost_with_cache:.2f}",
            'monthly_savings': f"${savings:.2f}",
            'savings_percentage': f"${savings_pct:.1f}%",
            'cache_cost': f"${self.cache_cost:.2f}",
            'roi': f"${roi:.0f}%",
            'payback_days': (self.cache_cost / (savings / 30))
        }

# Example calculations
scenarios = [
    ("Small App", 1000, 0.80),    # 1K req/day, 80% hit rate
    ("Medium App", 10000, 0.85), # 10K req/day, 85% hit rate
    ("Large App", 100000, 0.90), # 100K req/day, 90% hit rate
]

for name, requests, hit_rate in scenarios:
    roi = CacheROI(requests_per_day=requests)
    result = roi.calculate(cache_hit_rate=hit_rate)

    print(f"\n{name} (Hit Rate: {hit_rate*100}%)"
          f" Requests/month: {result['monthly_requests']:,}")
    print(f" Without cache: {result['cost_without_cache']}")
    print(f" With cache: {result['cost_with_cache']}")
    print(f" Savings: {result['monthly_savings']}
    ({result['savings_percentage']})")
    print(f" ROI: {result['roi']}")
    print(f" Payback: {result['payback_days']:.1f} days")

```

"""

Output:

Small App (Hit Rate: 80%)

Requests/month: 30,000

Without cache: \$540.00

With cache: \$158.00

Savings: \$382.00 (70.7%)

ROI: 664%

Payback: 3.9 days

Medium App (Hit Rate: 85%)

Requests/month: 300,000

Without cache: \$5,400.00

With cache: \$860.00

Savings: \$4,540.00 (84.1%)

ROI: 8980%

Payback: 0.3 days

Large App (Hit Rate: 90%)

Requests/month: 3,000,000

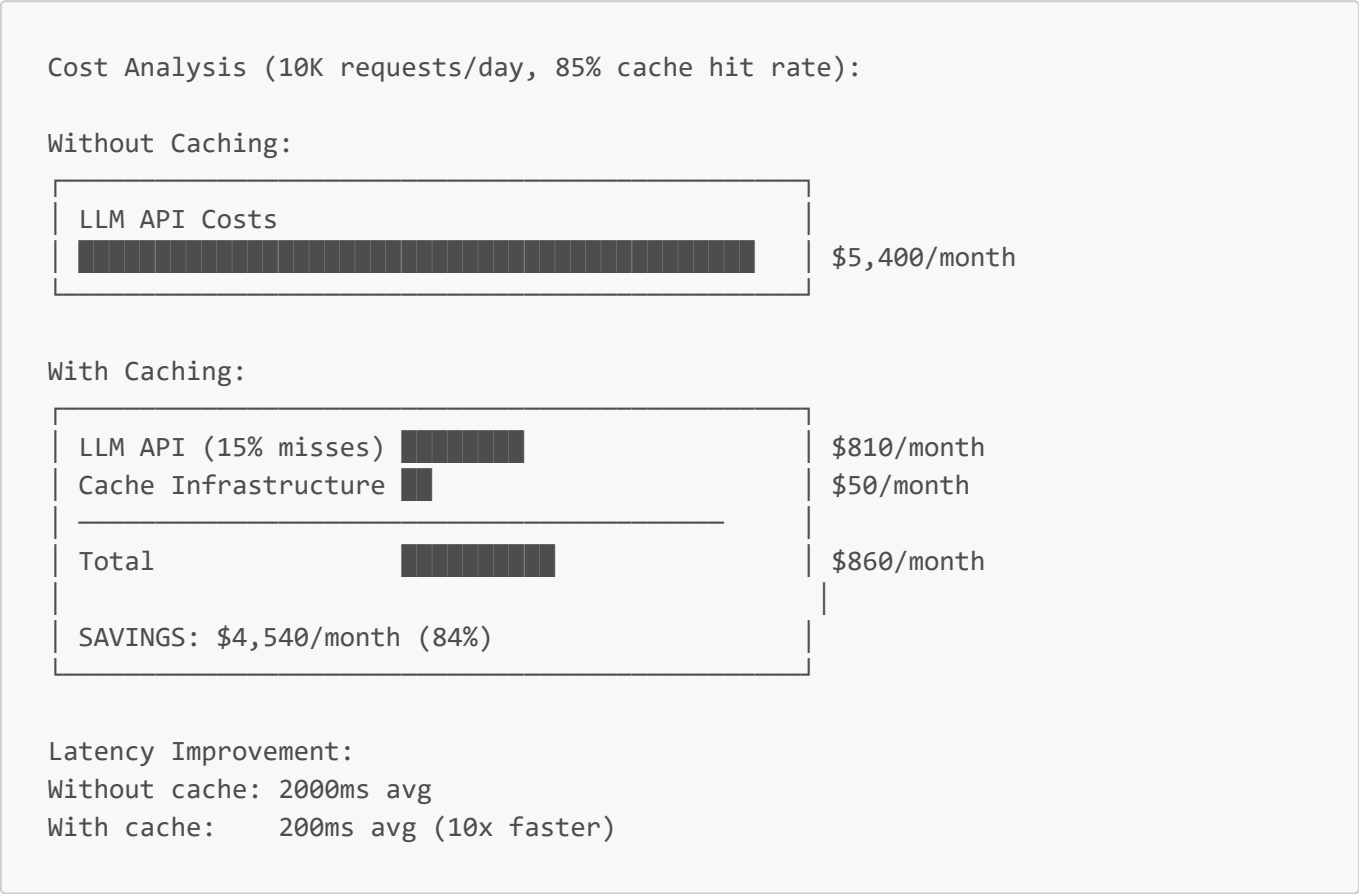
Without cache: \$54,000.00

With cache: \$5,450.00

Savings: \$48,550.00 (89.9%)

```
ROI: 97000%
Payback: 0.03 days (< 1 hour!)
"""
```

Cost Breakdown Visualization



8. Best Practices & Anti-patterns

What TO Cache ☒

- ☒ GOOD to cache:
1. Deterministic Responses:
 - Temperature = 0
 - Same input → Same output
 - Example: "What is 2+2?" → "4"

2. Frequently Asked Questions:
 - "What are your hours?"
 - "How do I reset password?"
 - High repeat rate

3. Expensive Operations:
 - Long prompts (>1000 tokens)
 - Complex reasoning tasks
 - Multi-step agents

4. Stable Data:
- Historical facts
 - Mathematical calculations
 - Code explanations
5. Embeddings:
- Document embeddings
 - Query embeddings
 - Feature vectors

What NOT to Cache ❌

- ❌ BAD to cache:
1. Non-deterministic Responses:
- Temperature > 0
 - Random/creative outputs
 - Example: "Write a random story"
2. Time-sensitive Data:
- "What time is it?"
 - "Current stock price"
 - "Today's weather"
3. Personalized Content:
- User-specific recommendations
 - Private information
 - Session-dependent data
4. Side Effects:
- Sending emails
 - Database writes
 - Payment processing
5. Frequently Changing:
- News articles
 - Live sports scores
 - Real-time data

Decision Matrix

Should I cache this?

Characteristic	Value	Cache	Notes
Deterministic	Yes	☑	Same in → Same out
Deterministic	No	❌	Random outputs

Data freshness	Static	☑	Historical facts
Data freshness	Dynamic	⚠	Use short TTL
Data freshness	Real-time	✗	Always fetch fresh
Repeat rate	High	☑	FAQ, common queries
Repeat rate	Low	✗	Unique queries
Cost per call	High	☑	Long prompts
Cost per call	Low	⚠	Consider overhead
Side effects	Yes	✗	Never cache
Side effects	No	☑	Pure functions

☑ = Recommended to cache

⚠ = Cache with caution (short TTL)

✗ = Do NOT cache

Security & Privacy

```
class SecureCacheWrapper:
    """Cache wrapper with security considerations"""

    def __init__(self, cache):
        self.cache = cache
        self.pii_detector = PIIDetector()

    def safe_cache(self, query: str, response: str) -> bool:
        """Only cache if safe"""

        # Check for PII in query and response
        if self.pii_detector.contains_pii(query):
            print("⚠ PII detected in query - NOT caching")
            return False

        if self.pii_detector.contains_pii(response):
            print("⚠ PII detected in response - NOT caching")
            return False

        # Check for sensitive keywords
        sensitive_keywords = ['password', 'ssn', 'credit card', 'api key']
        if any(kw in query.lower() for kw in sensitive_keywords):
            print("⚠ Sensitive content - NOT caching")
            return False

        # Safe to cache
        self.cache.set(query, response)
        return True

class PIIDetector:
```

```

        """Detect personally identifiable information"""

        def contains_pii(self, text: str) -> bool:
            import re

            # Email pattern
            if re.search(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b',
text):
                return True

            # Phone number
            if re.search(r'\b\d{3}[-.]?\d{3}[-.]?\d{4}\b', text):
                return True

            # SSN
            if re.search(r'\b\d{3}-\d{2}-\d{4}\b', text):
                return True

            # Credit card
            if re.search(r'\b\d{4}[- ]?\d{4}[- ]?\d{4}[- ]?\d{4}\b', text):
                return True

            return False

```

9. Real-world Use Cases

Use Case 1: RAG Chatbot with Multi-tier Cache

E-commerce Customer Support Bot:

- 10,000 queries/day
- 70% are FAQ (cacheable)
- 20% are product questions (vector cache)
- 10% are order-specific (not cached)

Architecture:

User → L1 (FAQ exact match) → L2 (Redis) → L3 (Semantic) → RAG + LLM

Results:

- Cache hit rate: 85%
- Avg latency: 150ms (vs 2000ms without cache)
- Cost: \$200/month (vs \$2,500 without cache)
- Savings: \$2,300/month (92%)

Use Case 2: Code Documentation Assistant

```

class CodeDocCache:
    """Cache for code documentation queries"""

```

```
def __init__(self):
    self.exact_cache = {} # Exact code → explanation
    self.semantic_cache = SemanticCache() # Similar code patterns
    self.embedding_cache = VectorCache() # Code embeddings

def explain_code(self, code_snippet: str) -> str:
    """Explain code with multi-level caching"""

    # Level 1: Exact match
    if code_snippet in self.exact_cache:
        return self.exact_cache[code_snippet]

    # Level 2: Semantic match (similar code)
    semantic_result = self.semantic_cache.find_similar(code_snippet,
threshold=0.90)
    if semantic_result:
        return semantic_result

    # Level 3: Generate explanation
    explanation = self._generate_explanation(code_snippet)

    # Cache at all levels
    self.exact_cache[code_snippet] = explanation
    self.semantic_cache.add(code_snippet, explanation)

    return explanation
```

Summary

Cache Type Comparison

Cache Type	Best For	Hit Rate	Latency	Cost Savings
Prompt Cache	Long system prompts	70-90%	N/A	90-95%
Response Cache	Exact queries	60-80%	<5ms	80-90%
Vector Cache	Embeddings	85-95%	<10ms	90-95%
Tool Cache	API calls, DB queries	70-85%	<50ms	70-85%
Semantic Cache	Similar queries	75-90%	<100ms	75-90%
Multi-tier	Production systems	85-95%	10-200ms	85-95%

Quick Decision Guide

Start here:

|

├─ Exact same queries repeated?

| └─ YES → Response Cache (Redis)

- └ Long system prompts?
 - └ YES → Prompt Cache (Anthropic/OpenAI)
- └ Similar but not identical queries?
 - └ YES → Semantic Cache (Vector DB)
- └ Embedding operations?
 - └ YES → Vector Cache (Redis/In-memory)
- └ Tool/API calls?
 - └ YES → Tool Cache (with smart invalidation)
- └ High traffic production?
 - └ YES → Multi-tier (L1 + L2 + L3)

Key Takeaways

1. **Always cache embeddings** - 95%+ hit rate, huge cost savings
2. **Cache deterministic responses** - temperature=0 only
3. **Use semantic caching for variations** - "What is X?" vs "Explain X"
4. **Multi-tier for production** - L1 (memory) → L2 (Redis) → L3 (vector)
5. **Monitor cache hit rates** - Aim for 80%+ hit rate
6. **Set appropriate TTLs** - Static data: long, dynamic data: short
7. **Never cache PII** - Security and privacy first
8. **Measure ROI** - Track costs and latency improvements

Expected Results:

- 80-95% cost reduction
- 10-100x latency improvement
- 85%+ cache hit rate (well-tuned system)
- ROI payback in days/weeks

Last updated: August 2026